

Supplementary Material

Quantized Model Adoption in Open-Source Software: Prevalence, Practices, and Evolution

Anonymous Author(s)

APPENDIX A DATA CONSTRUCTION DETAILS

This appendix gives the full procedural detail summarized in the Study Design (Sec. III).

A. Quantized-Model Detection and Taxonomy

This subsection gives the full detection catalogs, keyword sets, and normalization rules summarized in Sec. III-A.

Layer 1 catalog. L1 reads `config.quantization_config.quant_method` and matches it case-insensitively against a catalog assembled from neural-network and LLM-quantization surveys [1]–[6], augmented with families post-dating them. The catalog covers post-training methods (GPTQ [7], AWQ [8], LLM.int8()/BitsAndBytes [9], AQLM [10], QuIP/QuIP# [11], [12], HQQ [13], SqueezeLLM [14], VPTQ [15], SpQR [16], SmoothQuant [17], OmniQuant [18], OWQ [19], QuaRot [20], FlatQuant [21], ZeroQuant [22], RPTQ [23], PB-LLM [24]), low-bit formats (FP8 [25], MXFP4/MXFP8 [26], GGUF/GGML [27]), inference kernels (Marlin [28]), extreme-low-bit families (BitNet [29]), and rounding methods (AutoRound [30]), plus library-emitted labels (EETQ [31], Quanto [32], TorchAO [33], EXL2/EXL3 [34], GPTQModel [35], CompressedTensors [36]). We accept canonical labels and documented aliases (e.g., `bnb_4bit`, `nf4`), record auxiliary fields (`bits`, `group_size`, `quant_type`) to refine labels downstream, and tag unmatched `quant_method` strings `unknown_quantized` for later recovery.

Layer 2 signals. L2 matches the tags array against the keyword set {`gguf`, `ggml`, `gptq`, `awq`, `bnb`, `bitsandbytes`, `4-bit`, `8-bit`, `4bit`, `8bit`, `exl2`, `exl3`, `aqlm`, `quip`, `hqq`, `eetq`, `quantized`, `int4`, `int8`, `fp8`, `nf4`, `mxfp4`, `nvfp4`} and maps `library_name` to a method or ecosystem: `ctransformers` → GGUF/GGML; `peft` → PEFT/LoRA; `mlx/mlx-lm/mlx-vlm/mflux` → MLX; `mlc-llm` → MLC-LLM; `vllm` → vLLM-quantized (with a bit-width tag). Export-only frameworks (`onnx`, `openvino`, `tensorrt`, `coreml`) require a co-occurring quantization signal, since they also produce non-quantized exports.

Layer 3 heuristics. L3 matches (a) identifier patterns—quantization keywords as left-boundary tokens (boundary class `[-_./]`), including `gptq`, `awq`, `gguf`, `ggml`, `bnb`, `nf4`, `exl2`, `exl3`, `int2-int8`, `fp8`, `2bit-8bit`, `aqlm`, `hqq`, `vptq`, `marlin`, and others, plus K-quant patterns (e.g., `q4_k_m` → GGUF) and group-size patterns (e.g., `g128` → GPTQ); (b) `.gguf/.ggml` files in siblings (standalone GGUF/GGML signal); and

(c) quantization-config sibling files (`quantize_config.json`, `quant_config.json`, `quantization_config.json`).

Confidence classification. A model is *high-confidence quantized* when a signal names a specific quantization method, format, or precision: (i) an L1 `quant_method` match, (ii) an L2 tag or library label, or (iii) L3 evidence—a `.gguf/.ggml` file, a config sibling, or a K-quant/group-size pattern. Generic markers alone—a bare quantized tag, or an export-only framework tag without a co-occurring quantization signal—leave a model *candidate-only* and excluded. Of 275,306 detected models, 271,996 are high-confidence and 3,310 candidate-only. Of the high-confidence set, 89.6% are confirmed by two or more independent layers; the remainder rests on L3 alone (9.8%), L2 alone (0.5%), or L1 alone (0.1%), and 24.7% carry an L1 `quant_method` catalog match.

Normalization rules (priority order). (1) Drop detection-evidence tokens (`quantized_config_file`, `unknown_quantized`, etc.). (2) Recover a specific method for bit-width-only L1 hits by re-inspecting `library_name`, tags, and identifier (recovers MXFP4, MXFP8, AutoRound, NVFP4/ModelOpt, BitNet, Quark, HIGGS, and small leaks of named methods). (3) Collapse variants to a canonical label (K-quant → GGUF; group-size → GPTQ; all `BitsAndBytes` variants → `BitsAndBytes`). (4) Separate auxiliary-ecosystem labels (MLX, MLC-LLM, vLLM-quantized, ONNX, OpenVINO, TensorRT, CoreML) from the primary distribution. (5) Resolve bit-widths: drop when redundant with a named method, else relabel *Generic N-bit*. (6) Tag PEFT/LoRA adapters (`library_name=peft` or identifier in {`qlora`, `lora`, `dora`, `adapter`}) as non-method artifacts.

Tab. I enumerates all 62 labels across the five categories, with each label’s Hugging Face supply and analysis-set adoption.

B. GitHub Repository Mining

This subsection gives the full keyword sets, file-class definitions, and loader patterns for the five-stage mining pipeline summarized in Sec. III-B.

Stage 1: Code Search. We issued GitHub Code Search API queries for each model’s full Hugging Face identifier (i.e., `Organization/ModelName`) over public repository source code, unconstrained by programming language, searching the exact identifier as a quoted string to reduce false positives. For each match we recorded the repository name and file path. Subject to the API’s rate limits, the search was distributed across nine API tokens and ran over several weeks.

Stage 2: Precision filtering. Identifiers containing an explicit quantization keyword (GPTQ, AWQ, GGUF,

TABLE I

TAXONOMY OF QUANTIZED-MODEL LABELS. “HF” COUNTS HIGH-CONFIDENCE HUGGING FACE MODELS WITH THAT LABEL; “REPOS” COUNTS ANALYSIS-SET REPOSITORIES ($n = 22,024$) REFERENCING AT LEAST ONE; “—” MARKS LABELS WITHOUT A CANONICAL CITATION.

Method	HF	Repos	Ref.	Method	HF	Repos	Ref.
Algorithm methods (38 labels)				Algorithm methods (cont.)			
GGUF	167,006	10,869	[27]	QuaRot	7	0	[20]
BitsAndBytes	46,228	5,516	[9]	EETQ	6	26	[31]
EXL2	10,241	370	[34]	SpQR	2	175	[16]
GPTQ	9,568	4,243	[7]	ctransformers	1	74	[43]
AWQ	9,025	3,766	[8]	Precision-only (10 labels)			
CompressedTensors	4,817	1,264	[36]	INT4	3,956	1,092	—
GGML	1,555	1,649	[27]	FP8	3,612	2,695	[25]
EXL3	1,350	10	[34]	INT8	3,352	1,363	—
MXFP4	1,295	332	[26]	NF4	678	214	[44]
AutoRound	586	297	[30]	INT3	367	2	—
TorchAO	403	196	[33]	FP4	356	175	—
NVFP4/ModelOpt	356	193	[37]	INT2	322	1	—
HQQ	334	205	[13]	fbgemm-fp8	21	282	—
SmoothQuant	185	26	[17]	INT5	2	0	—
GPTQModel	172	243	[35]	INT6	1	0	—
AQLM	145	730	[10]	Auxiliary ecosystem (7 labels)			
ExLlamaV2	145	7	[34]	MLX	9,183	1,333	[45]
nnf_quantized	142	5	[38]	vLLM-quantized	2,206	1,333	[46]
BitNet	130	213	[29]	ONNX-quantized	1,257	183	[47]
Quark	129	389	[39]	OpenVINO	1,041	138	[48]
Marlin	102	207	[28]	MLC-LLM	104	2	[49]
VPTQ	99	196	[15]	CoreML	91	8	[50]
MXFP8	71	3	[26]	TensorRT	39	29	[51]
neural-comp._quant.	58	12	[40]	Generic-bitwidth (6 labels)			
QuIP	54	78	[11]	Generic 4-bit	1,797	879	—
optimum_quantized	29	16	[41]	Generic 8-bit	1,275	288	—
SqueezeLLM	28	0	[14]	Generic 2-bit	223	6	—
Quanto	23	4	[32]	Generic 3-bit	160	7	—
HIGGS	22	0	[42]	Generic 6-bit	4	0	—
ExLlama	21	4	[34]	Generic 5-bit	1	0	—
LLM_int8	16	2	[9]	PEFT / LoRA (1 label)			
OmniQuant	15	12	[18]	PEFT/LoRA	23,935	1,137	[52]
FlatQuant	14	0	[21]				
PB-LLM	14	2	[24]				

GGML, BnB, EXL2, INT4/INT8, FP4/FP8, NF4, AQLM, HQQ, K-quant patterns, group-size patterns) are *Tier 1* (self-confirming); those without are *Tier 2* (ambiguous). For each of the 8,091 *Tier 2* models that surfaced a Stage 1 result, we issued contextual queries pairing the identifier with quantization-context tokens (e.g., `load_in_4bit`, `BitsAndBytesConfig`, `from_quantized`, `AutoGPTQForCausalLM`, `AutoAWQForCausalLM`, `gguf`, `ctransformers`), and admitted a repository only when it appeared in at least one such result. This verified 9,157 repositories (4,089 not already covered by *Tier 1*); the 7,996 uncorroborated were discarded.

Stage 3: File-type usage filtering. We classify each matched file into one of four classes: *code* (e.g., `.py`, `.ipynb`, `.sh`, `.go`, `.rs`, `.cpp`, plus `Dockerfile` and `Makefile`); *config* (e.g., `.yaml`, `.toml`, `.cfg`, `.env`, plus quantization-specific JSON such as `quantize_config.json`); *doc* (e.g., `.md`, `.rst`, `.html`, `.csv`, plus catalog paths such as `model-zoo/`, `leaderboard/`, `gh-pages/`, `docs/models/`); and *ambiguous* (e.g., `requirements.txt`, generic `.json`). A repository is *Tier A* if it has at least one *code* match; the rest are *config-only* (*Tier B*, 1,415), *mention-only* (*Tier C*, 9,560), or *ambiguous* (*Tier D*, 2,248).

Stage 4: Loader-pattern confirmation. For each *Tier A*

repository we fetched up to 25 matched code and config files via the GitHub Contents API and required two co-occurring conditions: (a) the model identifier present in the file, and (b) at least one *loader pattern*. Loader patterns capture standard loading mechanisms in the transformers library [53] and adjacent toolchains: `from_pretrained`, `AutoModelForCausalLM.from_pretrained`, `AutoTokenizer.from_pretrained`, `pipeline(...model=)`, `BitsAndBytesConfig` [9], `load_in_4bit=True`, `load_in_8bit=True`, `snapshot_download`, `hf_hub_download`, `AutoGPTQForCausalLM.from_quantized` [7], `AutoAWQForCausalLM.from_quantized` [8], `ctransformers...from_pretrained`, `llama_cpp.Llama`, `.gguf` path references [27], and `llama-cli/llama.cpp` CLI invocations. For Jupyter notebooks we parse the JSON and scan only code cells so markdown narration does not inflate confirmations. A repository is *loader-confirmed* (*Tier A2*) if at least one fetched file satisfies both conditions; this confirms 22,874 repositories (78.1% of *Tier A*), with the remaining 6,418 retained only as a sensitivity cohort.

Stage 5: PEFT/LoRA-only exclusion. A repository whose every referenced model is a PEFT/LoRA adapter [44], [52], [54], [55] (tagged per Sec. III-A2, Rule 6) does not establish quantized-model usage, since PEFT artifacts are fine-tuning layers on an already-quantized base rather than quantization choices. We exclude such *PEFT-only* repositories while retaining any *Tier A2* repository that also references a non-PEFT model (primary-method, auxiliary-ecosystem, or generic-residual). Of 22,874 *Tier A2* repositories, 340 (1.49%) are PEFT-only and removed, yielding the Final dataset of 22,534.

Off-hub signal set. The off-hub search (Sec. III-C) uses 87 signals across two families. *Self-quantization signals* match producer toolchains invoked in-repo: `AutoGPTQForCausalLM` and `GPTQModel` (`GPTQ`), `AutoAWQForCausalLM` (`AWQ`), `convert_hf_to_gguf` and `llama-quantize` (`llama.cpp/GGUF` conversion), `from_exllamav2` (`ExLlamaV2`), `compressed_tensors`, `smoothquant`, `quark.torch` (`AMD-Quark`), and `int4_weight_only/int8_weight_only` (`torchao`), among others. *Ollama signals* match obtaining or serving a model through Ollama: `ollama pull`, `ollama create`, registry references (`registry.ollama.ai`, `ollama.com/library`), `Modelfile` FROM directives, and `.gguf` references.

APPENDIX B ANALYSIS METHODOLOGY DETAILS

This appendix gives the procedural detail deferred from the RQ methodology (Sec. III-E1–Sec. III-E3). No additional statistical machinery is involved; the metrics are direct counts, rates, and distributions.

A. Per-RQ Metric Definitions

Repository signals (RQ₁). All three signals are retrieved via the GitHub REST API: stars from `stargazers_count`; contributor count from `/contributors?per_page=1&anon=true` using the Link header’s `rel=last page` number when

available, treated as an API-derived approximation otherwise; and default-branch commit count analogously from `/commits?per_page=1&sha={default_branch}`. For each signal we report the median, mean, IQR, and the 90th, 95th, and 99th percentiles, alongside a bucketed distribution. The bucket cuts are: for *stars*, 0, 1, 2–4, 5–9, 10–49, 50–499, ≥ 500 ; for *contributors*, 1, 2–3, 4–9, ≥ 10 ; and for *commits*, 1, 2–10, 11–100, ≥ 101 . The cuts are chosen to surface the boundaries relevant to the engineered-subset sensitivity analysis (Sec. B-B).

Language extraction (RQ₁). Primary language is the GitHub-reported repository language (language field; primary_language on the hub side). Reference-file language maps the extension of each loader-evidence file to a language (e.g., `.py` $\rightarrow$ Python, `.ipynb` $\rightarrow$ Jupyter, `.ts/.tsx` $\rightarrow$ TypeScript, `.js` $\rightarrow$ JavaScript, `.cpp/.cc/.h` $\rightarrow$ C++, `.rs` $\rightarrow$ Rust, `.go` $\rightarrow$ Go, `.sh` $\rightarrow$ Shell); extensionless files—common in off-hub Ollama configurations—are unmapped and excluded from the reference-file distribution, which lowers off-hub reference-file coverage (76.51%) relative to the hub (99.10%). A repository’s primary and reference-file languages diverge when its model-loading code is written in a different language from the bulk of the project.

Topic coding (RQ₁). Project purpose is read from the GitHub topics array, fetched via the repository REST endpoint for both channels. Topics are optional author-supplied metadata, so coverage is partial (12.21% of hub and 14.52% of off-hub repositories carry at least one topic); the purpose distribution is therefore reported over the tagged subset only and not generalized to the full population. We rank topics by the number of repositories carrying them and report the most frequent per channel without further normalization or merging of near-synonyms.

Prevalence and method mix (RQ₂). Restricting to models with at least one primary method label, we report: (i) the number of analysis-set repositories referencing at least one such model; (ii) the fraction of high-confidence Hugging Face quantized models appearing in at least one such repository; (iii) the fraction of repositories referencing more than one model; and (iv) the per-method distribution, both *by repository* (each repository contributing at most one count per method) and *by reference* (raw model-reference counts). Each method’s *adoption rate* is the number of its Hugging Face models appearing in at least one analysis-set repository divided by its total Hugging Face supply. We also report the distribution of distinct models per repository and the top publishers by distinct referencing repositories. Methods with fewer than 50 analysis-set repositories are grouped into an *Other* bucket in paper-facing tables; the full ranking is in the replication package [56]. Auxiliary-ecosystem labels (Sec. III-A2, Rule 4) and Generic N-bit residuals are excluded from the primary distribution; for each we separately report the number of Hugging Face models, referenced models, and analysis-set repositories, plus the fraction of repositories referencing at least one such model and the fraction referencing only such models. Primary outputs are computed under *primary scope*; a parallel *all-referenced scope* (every referenced model) is retained in the artifacts, with

the scope of each output documented there.

Bit-width recovery (RQ₂). The numerical precision of an adopted model is recovered along three paths applied in priority order: (a) an explicit bit-width-carrying label in the model’s quant_methods or tags (e.g., INT4, NF4, FP8, 4bit, GGUF K-quant tags such as Q4_K_M); (b) a config.quantization_config.bits value in the detection signals; and (c) a bit-width suffix in the model identifier (e.g., -bnb-4bit, -GPTQ-4bit-128g, -Q8_0). Bit-width is recoverable for 6,774 of the 13,754 distinct adopted hub models (49.26%); path (a) supplies the large majority (6,269), path (c) a further 505, and path (b) adds none beyond (a), since every model with a bits entry already carries a categorical label. Models with a method but no resolvable precision are excluded rather than imputed, so the reported distribution is over the recoverable subset only. Off-hub bit-width is recovered from the same precision-bearing signals (GGUF quant tags, Ollama quant tags, identifier suffixes); because off-hub artifacts are predominantly signal occurrences rather than distinct model identifiers, off-hub precision is reported as directional evidence only (Sec. C-A).

Temporal trends (RQ₃). First-use extraction clones each repository and searches its commit history for the addition of the model identifier, following established commit-mining guidance [57], [58]; for repositories referencing multiple models we track each introduction separately and record the earliest as the repository’s first adoption point. Aggregating first-use events yields per-year adoption counts. Adoption lag is computed only for referenced models carrying a Hugging Face creation timestamp. The off-hub channel uses the same procedure keyed on off-hub adoption signals (Sec. III-C) rather than hub identifiers.

B. Engineered-Project Sensitivity Analysis

The analysis set spans a wide range of repository maturity, from large, actively maintained projects to small single-commit experiments. Rather than impose a contested “engineered project” threshold as a hard filter—which would discard genuine instances of quantized-model usage and pre-empt the very distributional question RQ₁ examines—we retain all repositories in the primary analysis and instead report a sensitivity analysis. We recompute each RQ’s findings not only on the full analysis set but also on *engineered subsets* that exclude repositories flagged as disabled, template, single-commit, single-contributor, or below a star floor, following engineered-project signals from prior work [59], [60]. Because no canonical star threshold exists, we use three floors—a strict ≥ 10 matching the SEART GitHub Search Engine [61], an intermediate ≥ 5 , and a permissive ≥ 1 —and trace how the headline findings vary across them, making robustness to repository quality explicit while keeping the primary findings grounded in the full population of confirmed usage.

TABLE II

OFF-HUB BIT-WIDTH DISTRIBUTION FROM SIGNAL-OCCURRENCE EVIDENCE, SPLIT INTO THE LLM-SERVING STACK AND THE GENERAL-ML SUBSET (MUTUALLY EXCLUSIVE; 0 OVERLAPPING ARTIFACTS). COUNTS ARE PRECISION-BEARING ARTIFACTS (97.66% SIGNAL OCCURRENCES, 2.34% DISTINCT MODEL IDENTIFIERS), NOT STRICTLY COMPARABLE TO THE HUB’S MODEL-LEVEL COUNTS.

Bit-width	LLM-serving stack		General-ML	
	Count	%	Count	%
2-bit	8	1.48%	2	0.16%
3-bit	9	1.67%	2	0.16%
4-bit	199	36.85%	259	20.65%
5-bit	17	3.15%	0	0.00%
6-bit	7	1.30%	1	0.08%
8-bit	89	16.48%	780	62.20%
FP8	22	4.07%	69	5.50%
FP16	162	30.00%	57	4.55%
other (F32)	27	5.00%	84	6.70%
Total	540	100%	1,254	100%

APPENDIX C ADDITIONAL RESULTS

A. Additional RQ_2 Results: Off-Hub Bit-Width Distribution

Tab. II reports the off-hub bit-width distribution split into the LLM-serving stack and the general-ML subset (Sec. ??). The unit is a precision-bearing artifact recovered from off-hub signals; of the 1,794 such artifacts, only 42 (2.34%) resolve to a distinct model identifier, while 1,752 (97.66%) are per-repo signal occurrences deduplicated within, but not across, repositories. We therefore treat this distribution as a signal-occurrence proxy rather than a strict model-level count, and do not compare its magnitudes directly against the hub’s model-level counts (Tab. ??). Bit-width is recoverable for 1,116 of the 3,830 obtain-or-produce repositories (29.14%).

The two subsets diverge cleanly: the LLM-serving stack leads at 4-bit (36.85%), with a large FP16 share (30.00%) attributable to BF16/F16-tagged wrappers around GGUF artifacts that the hub side filters at the identifier level—excluding those wrappers raises the LLM-stack 4-bit share to roughly 52%. The general-ML subset is overwhelmingly 8-bit (62.20%), the ONNXRuntime/OpenVINO/Intel-NC integer-deployment pattern. This split is what drives the apparent 8-bit dominance in the unsplit off-hub cohort, and confirms that LLM-serving quantization converges toward 4-bit on both channels.

B. Additional RQ_2 Results: Adoption Share by Method

Fig. 1 reports each primary method’s share of all adopted models, complementing the adoption-rate view in Sec. IV-B.

C. Additional RQ_2 Results: Method Distribution Across Engineered Subsets

This subsection gives the full per-method breakdown across the four cohorts summarized in Sec. ?? . Tab. III reports the top 10 methods across the full analysis set and the ≥ 1 , ≥ 5 , and ≥ 10 -star engineered subsets.

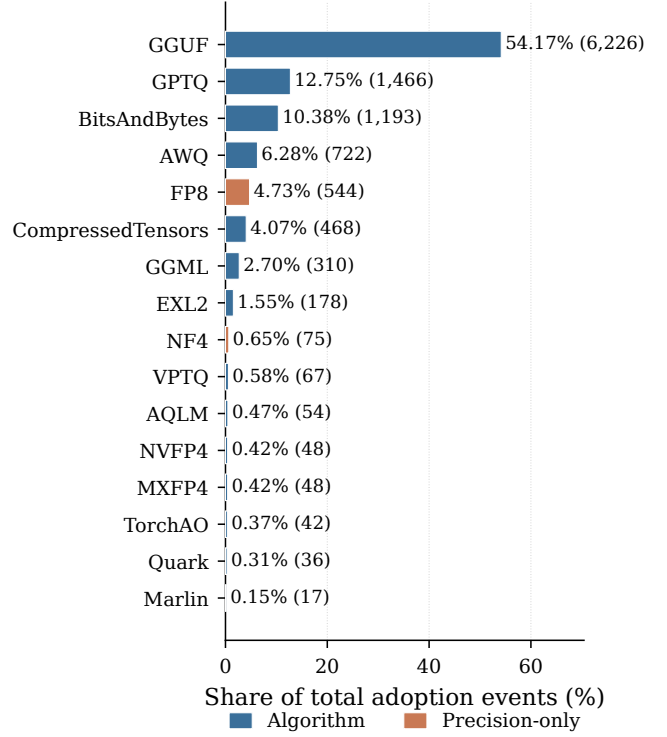


Fig. 1. Each primary quantization method’s percentage of all adopted models in the analysis set, sorted descending. The number in parentheses next to each bar is the absolute count of adopted HuggingFace models of that method (sum across methods = 11,494; the gap from the 13,754 distinct adopted models reflects adopted models whose only label is auxiliary-ecosystem, generic-bit-width, or PEFT, reported separately). Bars are colored by category: algorithm methods (blue) and precision-only labels (orange). NVFP4 denotes the NVFP4/ModelOpt family.

Method rankings are stable across the popularity range. GGUF, BitsAndBytes, GPTQ, and AWQ retain ranks 1–4 in every cohort, and the remaining six methods stay within the top 11 throughout. The dominance of these families is therefore not an artifact of the long tail of unnoticed solo repositories; they remain the most-adopted even after the strictest engineered cut removes roughly 90% of the analysis set.

EXL2 and AutoRound run the trend in reverse. Most algorithm methods gain percentage when small repositories are filtered out, but two move the opposite way: EXL2 drops from 1.68% in the full analysis set to 1.47% at ≥ 10 stars, and AutoRound from 1.35% to 0.89%. Each is tightly coupled to a single inference toolchain used predominantly by individual users—EXL2 to ExLlamaV2, AutoRound to Intel’s Neural Compressor—so the engineered filter trims exactly their core population, making them less visible without adoption actually shrinking. The broader pattern is symmetric: methods optimized for production serving frameworks (GPTQ, AWQ, FP8, CompressedTensors) gain percentage because their consuming projects are popular, while methods coupled to single-user inference libraries (EXL2, AutoRound, BitsAndBytes) lose percentage for the same reason.

Publisher rankings are stable but reorder at the top.

TheBloke, unsloth, Qwen, and bartowski occupy the top four in every cohort, but Qwen overtakes TheBloke for rank 1 at the ≥ 5 and ≥ 10 -star floors—consistent with Qwen’s own quantized variants attaching to a popular upstream family that accumulates stars, whereas TheBloke’s repackagings are widely used but predominantly in lower-popularity personal projects.

Auxiliary-ecosystem and PEFT shares are stable. Across all four cohorts, auxiliary-ecosystem coverage stays in a 12.7%–18.3% band, the exclusively-auxiliary share around 4%, and PEFT references in 5.2%–7.0% of repositories with PEFT $\cap$ BnB co-occurrence at 50.8%–53.3%. These are structural features of the dataset rather than long-tail artifacts.

D. Additional RQ_3 Results: Adoption Lag by Method

Tab. IV reports the full adoption-lag distribution per method summarized in Sec. IV-C2.

REFERENCES

- [1] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” in *Low-power computer vision*. Chapman and Hall/CRC, 2022, pp. 291–326.
- [2] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. Van Baalen, and T. Blankevoort, “A white paper on neural network quantization,” *arXiv preprint arXiv:2106.08295*, 2021.
- [3] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, “A survey on model compression for large language models,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 1556–1577, 2024.
- [4] R. Gong, Y. Ding, Z. Wang, C. Lv, X. Zheng, J. Du, Y. Yong, S. Gu, H. Qin, J. Guo, D. Lin, M. Magno, and X. Liu, “A survey of low-bit large language models: Basics, systems, and algorithms,” *Neural Networks*, vol. 192, p. 107856, 2025.
- [5] J. Zhao, M. Wang, M. Zhang, Y. Shang, X. Liu, Y. Wang, M. Zhang, and L. Nie, “Benchmarking post-training quantization in llms: Comprehensive taxonomy, unified evaluation, and comparative analysis,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.13178>
- [6] R. Jin, J. Du, W. Huang, W. Liu, J. Luan, B. Wang, and D. Xiong, “A comprehensive evaluation of quantization strategies for large language models,” in *Findings of the Association for Computational Linguistics: ACL 2024*, Aug. 2024, pp. 12 186–12 215.
- [7] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [8] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “Awq: Activation-aware weight quantization for on-device llm compression and acceleration,” *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.
- [9] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale,” *Advances in neural information processing systems*, vol. 35, pp. 30 318–30 332, 2022.
- [10] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh, “Extreme compression of large language models via additive quantization,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML’24, 2024.
- [11] J. Chee, Y. Cai, V. Kuleshov, and C. M. De Sa, “Quip: 2-bit quantization of large language models with guarantees,” *Advances in neural information processing systems*, vol. 36, pp. 4396–4429, 2023.
- [12] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, “Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks,” *Proceedings of machine learning research*, vol. 235, p. 48630, 2024.
- [13] H. Badri and A. Shaji, “Half-quadratic quantization of large machine learning models,” https://dropbox.github.io/hqq_blog/, 2023.
- [14] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, “Squeezellm: Dense-and-sparse quantization,” *arXiv preprint arXiv:2306.07629*, 2023.
- [15] Y. Liu, J. Wen, Y. Wang, S. Ye, L. L. Zhang, T. Cao, C. Li, and M. Yang, “Vptq: Extreme low-bit vector post-training quantization for large language models,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 8181–8196.
- [16] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, “Spqr: A sparse-quantized representation for near-lossless llm weight compression,” *arXiv preprint arXiv:2306.03078*, 2023.
- [17] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *International conference on machine learning*. PMLR, 2023, pp. 38 087–38 099.
- [18] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, “Omniquant: Omnidirectionally calibrated quantization for large language models,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=8Wuvhh0LYW>
- [19] C. Lee, J. Jin, T. Kim, H. Kim, and E. Park, “OWQ: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models,” in *AAAI Conference on Artificial Intelligence*, 2024.
- [20] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, “Quarot: Outlier-free 4-bit inference in rotated llms,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 100 213–100 240, 2024.
- [21] Y. Sun, R. Liu, H. Bai, H. Bao, K. Zhao, Y. Li, J. Hu, X. Yu, L. Hou, C. Yuan *et al.*, “Flatquant: Flatness matters for llm quantization,” *arXiv preprint arXiv:2410.09426*, 2024.
- [22] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [23] Z. Yuan, R. Liu, J. Cao, Y. Bondarenko, T. Blankevoort, B. Chen, and G. Sun, “RPTQ: Reorder-based post-training quantization for large language models,” *arXiv preprint arXiv:2304.01089*, 2023.
- [24] Y. Shang, Z. Yuan, Q. Wu, and Z. Dong, “Pb-llm: Partially binarized large language models,” *arXiv preprint arXiv:2310.00034*, 2023.
- [25] P. Micikevicius, D. Stolic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu *et al.*, “Fp8 formats for deep learning,” *arXiv preprint arXiv:2209.05433*, 2022.
- [26] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf *et al.*, “Microscaling data formats for deep learning,” *arXiv preprint arXiv:2310.10537*, 2023.
- [27] G. Gerganov, “GGUF file format specification,” [ggml-org/ggml](https://github.com/ggml-org/ggml), GitHub, 2023, accessed: 2026-06-05. [Online]. Available: <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>
- [28] E. Frantar, R. L. Castro, J. Chen, T. Hoefler, and D. Alistarh, “Marlin: Mixed-precision auto-regressive parallel inference on large language models,” in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2025, pp. 239–251.
- [29] H. Wang, S. Ma, L. Ma, L. Wang, W. Wang, L. Dong, S. Huang, H. Wang, J. Xue, R. Wang *et al.*, “Bitnet: 1-bit pre-training for large language models,” *Journal of Machine Learning Research*, vol. 26, no. 125, pp. 1–29, 2025.
- [30] W. Cheng, W. Zhang, H. Shen, Y. Cai, X. He, L. Kaokao, and Y. Liu, “Optimize weight rounding via signed gradient descent for the quantization of LLMs,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 11 332–11 350.
- [31] NetEase Youdao, “EETQ: Easy and efficient quantization for transformers,” <https://github.com/NetEase-FuXi/EETQ>, 2024, accessed: 2026-06-05.
- [32] HuggingFace, “Quanto: A pytorch quantization toolkit,” <https://github.com/huggingface/optimum-quanto>, 2024, accessed: 2026-06-05.
- [33] PyTorch Team, “torchao: Pytorch native quantization and sparsity for training and inference,” <https://github.com/pytorch/ao>, 2024, accessed: 2026-06-05.
- [34] turboderp, “ExLlamaV2: A fast inference library for running LLMs locally on modern consumer-class GPUs,” <https://github.com/turboderp/exllamav2>, 2024, accessed: 2026-06-05.
- [35] ModelCloud, “GPTQModel: Production-ready LLM quantization toolkit based on the GPTQ algorithm,” <https://github.com/ModelCloud/GPTQModel>, 2024, accessed: 2026-06-05.

TABLE III

TOP 10 PRIMARY QUANTIZATION METHODS (BY ANALYSIS-SET RANK) ACROSS THE FOUR COHORTS. *Other* AGGREGATES EIGHT SMALLER METHODS (AQLM, QUARK, EXL2, MXFP4, AUTOROUND, GENERIC 8-BIT, FBGEMM-FP8, NF4).

	Analysis set $n = 22,024$		≥ 1 star $n = 4,861$		≥ 5 stars $n = 2,831$		≥ 10 stars $n = 2,246$	
Method	Repos	%	Repos	%	Repos	%	Repos	%
GGUF	10,869	49.35%	2,542	52.29%	1,447	51.11%	1,137	50.62%
BitsAndBytes	5,516	25.05%	1,027	21.13%	580	20.49%	450	20.04%
GPTQ	4,243	19.27%	1,082	22.26%	710	25.08%	582	25.91%
AWQ	3,766	17.10%	1,030	21.19%	680	24.02%	554	24.67%
FP8	2,695	12.24%	858	17.65%	587	20.73%	498	22.17%
GGML	1,649	7.49%	341	7.02%	197	6.96%	161	7.17%
INT8	1,363	6.19%	300	6.17%	202	7.14%	171	7.61%
CompressedTensors	1,264	5.74%	412	8.48%	270	9.54%	219	9.75%
INT4	1,092	4.96%	306	6.30%	205	7.24%	174	7.75%
Generic 4-bit	879	3.99%	219	4.51%	157	5.55%	132	5.88%
Other (8 methods)	2,902	13.18%	749	15.41%	483	17.06%	394	17.54%

TABLE IV

ADOPTION LAG (DAYS) FROM A MODEL'S PUBLICATION TO ITS FIRST APPEARANCE IN A REPOSITORY, PER METHOD. THE MEDIAN'S BOOTSTRAP 95% CI USES 10,000 RESAMPLES. "OTHER" AGGREGATES THE REMAINING LOWER-VOLUME METHODS.

Method	n	Median	95% CI	IQR
BitsAndBytes	7,483	175.2	[170.1, 181.1]	71.6–335.8
GGUF	13,783	202.1	[196.8, 207.8]	78.5–413.3
AWQ	2,716	239.2	[227.5, 250.5]	107.5–403.3
GPTQ	16,230	429.1	[423.8, 435.7]	206.6–586.2
Other	4,768	172.2	[165.1, 179.8]	54.6–335.4

- [36] Neural Magic, "compressed-tensors: A safetensors extension for efficient storage of quantized models," <https://github.com/neuralmagic/compressed-tensors>, 2024, accessed: 2026-06-05.
- [37] NVIDIA, "Pretraining large language models with nvfp4," 2026. [Online]. Available: <https://arxiv.org/abs/2509.25149>
- [38] A. Kozlov, I. Lazarevich, V. Shamporov, N. Lyalyushkin, and Y. Gorbachev, "Neural network compression framework for fast model inference," in *Intelligent Computing: Proceedings of the 2021 Computing Conference, Volume 3*. Springer, 2021, pp. 213–232.
- [39] Advanced Micro Devices, "AMD Quark: A cross-platform deep learning model optimization toolkit," <https://github.com/amd/Quark>, 2025, accessed: 2026-06-05.
- [40] F. Tian, H. Chang, H. Shen, and S. Chen, "Intel Neural Compressor," <https://github.com/intel/neural-compressor>, 2022, accessed: 2026-06-05.
- [41] HuggingFace, "Optimum: Hardware optimization tools for Transformers, Diffusers, TIMM and Sentence Transformers," <https://github.com/huggingface/optimum>, 2023, accessed: 2026-06-05.
- [42] V. Malinovsky, A. Panferov, I. Ilin, H. Guo, P. Richtárik, and D. Alistarh, "Higgs: Pushing the limits of large language model quantization via the linearity theorem," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025, pp. 10 857–10 886.
- [43] "ctransformers: Python bindings for the Transformer models implemented in C/C++ using the GGML library," <https://github.com/marella/ctransformers>, 2023, accessed: 2026-06-05.
- [44] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in neural information processing systems*, vol. 36, pp. 10 088–10 115, 2023.
- [45] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert, "Mlx: Efficient and flexible machine learning on apple silicon, 2023," URL <https://github.com/ml-explore>.
- [46] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.
- [47] Microsoft, "ONNX Runtime: cross-platform, high performance ML inferencing and training accelerator," <https://github.com/microsoft/onnxruntime>, 2018, accessed: 2026-06-05.
- [48] Intel, "OpenVINO toolkit," <https://github.com/openvinotoolkit/openvino>, 2018, accessed: 2026-06-05.
- [49] M. Mlc-llm, "Universal llm deployment engine with ml compilation," <https://github.com/mlc-ai/mlc-llm>, 2023.
- [50] Apple, "Core ML: Integrate machine learning models into your app," <https://developer.apple.com/documentation/coreml>, 2017, accessed: 2026-06-05.
- [51] NVIDIA, "TensorRT: high-performance deep learning inference SDK," <https://github.com/NVIDIA/TensorRT>, 2017, accessed: 2026-06-05.
- [52] E. J. Hu, yelong shen, P. Sanh, J. Chaumond, C. Delangue, A. Moi, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [53] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.
- [54] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen, "Dora: Weight-decomposed low-rank adaptation," in *Forty-first International Conference on Machine Learning*, 2024.
- [55] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang, "Parameter-efficient fine-tuning for large models: A comprehensive survey," *arXiv preprint arXiv:2403.14608*, 2024.
- [56] Anonymous Author(s), "Replication package: Quantized model adoption in open-source software: Prevalence, practices, and evolution," <https://github.com/projectHF2026/project-hf-quantization-mining>, 2026, anonymized for review.
- [57] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining github," in *Proceedings of the 11th working conference on mining software repositories*, 2014, pp. 92–101.
- [58] T. Kinsman, M. Wessel, M. A. Gerosa, and C. Treude, "How do software developers use github actions to automate their workflows?" in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 2021, pp. 420–431.
- [59] N. Munaiah, S. Kroh, C. Cabrey, and M. Nagappan, "Curating github for engineered software projects," *Empirical Software Engineering*, vol. 22, no. 6, pp. 3219–3253, 2017.
- [60] P. Pickerill, H. J. Jungen, M. Ochodek, M. Maćkowiak, and M. Staron, "Phantom: Curating github for engineered software projects using time-series clustering," *Empirical Software Engineering*, vol. 25, no. 4, pp. 2897–2929, 2020.
- [61] O. Dabic, E. Aghajani, and G. Bavota, "Sampling projects in GitHub for MSR studies," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 2021, pp. 560–564.